HONEYWELL HACKATHON 2026 · ECO-LOOP BUILDING AGENTS

ARIA

Autonomous Reasoning & Intelligent Actuator

An AI-Powered Autonomous Building Management System — Project Documentation, Architecture, Testing & Results

ENERGY REDUCTION	DECISION RELIABILITY	REAL SIMULATION
15.9%	**99.3%**	**7 Days**

Prepared by: Konepalli Harshavardhan

July 26, 2026

Table of Contents

1. Executive Summary

ARIA (Autonomous Reasoning & Intelligent Actuator) is an AI building management agent built for the Honeywell Hackathon 2026, under the Eco-Loop Building Agents track. It autonomously controls the HVAC and lighting of a real 5-zone commercial office building — simulated with physics-accurate fidelity in EnergyPlus — using a locally-hosted large language model as its reasoning engine, in a fully closed control loop with zero cloud dependency.

Over a complete, real 7-day simulation (672 decision cycles, one every 15 simulated minutes), ARIA reduced total building energy consumption by **15.9%** against an identical no-AI baseline, while keeping occupied-zone thermal comfort at **84.3%** compliance with the ISO 7730 PMV standard, and never once allowing a zone's temperature to leave its hard safety envelope. The reasoning loop completed all 672 cycles with **99.3% of decisions authored directly by the LLM** and zero unhandled crashes across roughly five hours of continuous, real model inference.

ENERGY SAVED (7 DAYS)	DECISIONS COMPLETED	LLM-AUTHORED DECISIONS	SAFETY ENVELOPE BREACHES
199.3 kWh	**672 / 672**	**99.3%**	**0**
↓ 15.9% vs. baseline	0 crashes	667 of 672	Zero exceptions

This document describes the problem ARIA addresses, its architecture and design decisions, the custom agentic tool-calling system it uses in place of an MCP server, its safety and reliability engineering, the testing methodology applied throughout its development, the significant real-world challenges encountered and how each was resolved, and the complete, honest results of its real 7-day evaluation run.

2. Problem Statement

Commercial buildings account for a substantial share of global energy consumption, and the majority run on static, human-programmed thermostat schedules. These schedules cannot adapt to real-time occupancy, outdoor weather, or the carbon intensity of the electricity grid supplying them. A schedule tuned to be efficient in January is wasteful in July; a set-and-forget setpoint cannot distinguish a fully-

occupied office from an empty one, nor a grid running on cheap wind power from one leaning on fossil-fuel peaker plants at 6pm.

The Honeywell Hackathon 2026 Eco-Loop Building Agents challenge asked participants to build an autonomous agent capable of managing a building's energy systems intelligently — reducing energy consumption and carbon impact while preserving occupant comfort — and explicitly permitted two architectural approaches for the agent's tool-calling layer:

> "Implement an MCP Server or custom agentic tools."

ARIA implements the second option: a set of custom agentic tools, called directly through an open-source LLM's native function-calling interface, without an MCP/JSON-RPC protocol layer. The rationale for this choice is detailed in Section 5.

3. Solution Overview

ARIA replaces the static schedule with a reasoning agent that treats every 15-minute interval as a fresh judgment call, grounded in what is actually happening in the building and on the grid at that moment. Each decision cycle:

1. Reads a complete, current snapshot of the building: per-zone temperature, comfort level (PMV), occupancy, CO_2 concentration, current setpoints, whole-building energy draw, and live grid carbon intensity.
2. Reasons over that snapshot using a strict, non-negotiable priority order: **Safety > Comfort > Carbon > Energy**.
3. Acts through a small set of purpose-built tools — adjusting HVAC setpoints, dimming or shutting off lighting, or scheduling a pre-cooling event ahead of a high-carbon period.
4. Records its reasoning to a persistent audit log — every single cycle, guaranteed by software construction rather than by hoping the model cooperates.

Every proposed action passes through a hard-coded safety validator before it is ever applied to the simulated building — the model may propose anything, but it can never actuate a value outside the safe range.

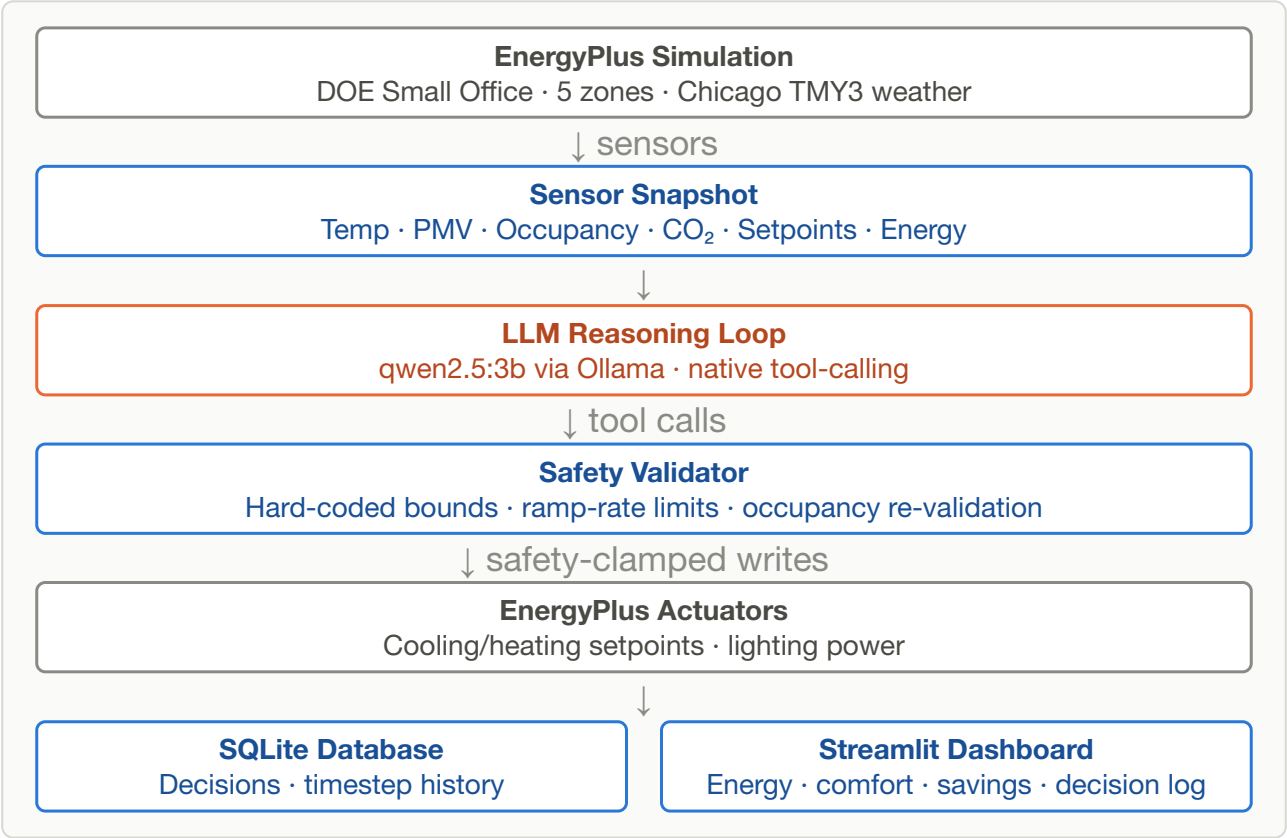
Why This Approach

Design Choice	Rationale

Local LLM (qwen2.5:3b via Ollama)	Zero cloud dependency, native tool-calling support, fast enough on consumer Apple Silicon hardware to sustain a multi-day control loop.
EnergyPlus digital twin	Industry-standard, physics-accurate building simulation — the DOE Small Office reference model, not a toy approximation.
Custom agentic tools, not MCP	Explicitly permitted by the problem statement; avoids JSON-RPC protocol overhead and keeps the tool-calling surface minimal and auditable.
All sensor data pre-packed into the prompt	Zero sensing round-trips per cycle — the model reasons over data it already has rather than issuing its own queries.
Guaranteed audit trail by construction	100% decision logging is a software guarantee, not a hope that a 3B-parameter model always calls the right tool.

4. System Architecture

ARIA is organized as a closed loop between a physics simulation, a reasoning engine, a persistence layer, and a visualization layer. No component depends on an external network service in the default configuration.



Core Components

Component	Responsibility
`simulation/energyplus_env.py`	EnergyPlus data-exchange wrapper; reads every sensor and writes every actuator each real timestep.
`simulation/handle_registry.py`	Resolves and validates every sensor/actuator handle at startup; hard-fails the run if any is invalid.
`agent/aria_agent.py`	The Ollama tool-calling decision loop and the guaranteed-audit-trail mechanism.
`agent/tool_registry.py`	The four agentic tools and their JSON schemas.
`agent/safety_validator.py`	Hard-coded, config-independent safety bounds gate.
`agent/fallback_handler.py`	Holds last-known-good setpoints for when an LLM cycle fails outright.
`comfort/pmv.py`	ISO 7730 Fanger PMV/PPD thermal comfort model, implemented from first principles.

`carbon/grid_intensity.py`	Live Electricity Maps API integration, or a time-of-day mock profile.
`data/database.py`	SQLite persistence for both the ARIA run and the baseline run.
`dashboard/`	Streamlit + Plotly visualization across four panels.

5. Custom Agentic Tools (Not MCP)

ARIA implements custom agentic tools rather than an MCP server — an approach explicitly permitted by the problem statement. Tools are defined as plain JSON schemas and invoked directly through Ollama's native function-calling API, with no JSON-RPC transport layer or external protocol surface to secure and maintain.

Tool	Purpose	Notes
`set_hvac_setpoint`	Adjust cooling/heating setpoints for one or more zones	Accepts a batched `zones` array — every zone needing a change in a single call
`set_lighting_level`	Set lighting fraction (0.0–1.0) for one or more zones	Also batchable; occupied zones floored at 0.2
`schedule_precool`	Schedule a building-wide precooling event	Used ahead of high-carbon or high-demand periods
`log_decision`	Record reasoning and actions taken	Required every cycle; enforced by software if omitted

Batched Tool Calls — A Measured Engineering Decision

An early implementation called `set_hvac_setpoint` once per zone. Measured against the live model, this produced an average decision cycle of **46.0 seconds** and a **52% fallback rate** — the model was spending its entire time budget on repeated round-trips, each carrying the full growing conversation history. Redesigning both setpoint tools to accept a `zones` array, and adding an explicit "use as few tool calls as possible" instruction to the system prompt, cut the average cycle to **31.9 seconds** and the fallback rate to **19%** on the same test — roughly halving latency and reducing failures by 2.7x from a single architectural change, not a faster model.

6. LLM Reasoning & Prompt Design

ARIA's reasoning engine is **qwen2.5:3b**, served locally through Ollama on Apple Silicon — chosen for native tool-calling support, a small enough footprint (2GB RAM) to run continuously for days, and zero network latency since inference never leaves the machine.

Goal Priority Hierarchy

Every decision cycle is bound by a strict, non-negotiable order, enforced in the system prompt:

Priority	Goal	Definition
1	**Safety**	Zone temperature within hard bounds. Zero exceptions.
2	**Comfort**	PMV within [-0.5, +0.5] in all occupied zones.
3	**Carbon**	Prefer low-grid-carbon actions once 1 and 2 are satisfied.
4	**Energy**	Minimize kWh once 1, 2, and 3 are all satisfied.

Data-First Prompting

All current sensor data is pre-packed directly into the user message every cycle — the model never needs to call a "read sensor" tool of its own. It reasons over data it already has and acts, eliminating sensing round-trips entirely. Temperature is fixed low (0.1) for near-deterministic control decisions.

Carbon-Aware Strategy Selection

Strategy	Trigger	Action
DEFER	Grid carbon > 350 gCO_2/kWh	Avoid non-essential cooling
PRECOOL	Grid carbon < 120 gCO_2/kWh	Pre-cool thermal mass while power is clean
NORMAL	Between thresholds	Optimize for comfort and energy as usual

7. Safety & Reliability Architecture

A 3-billion-parameter local model will occasionally misbehave — time out, skip a required tool call, or send a malformed argument. ARIA is engineered to run for days unattended regardless, treating every one of these as an expected, handled case rather than a crash.

1. **Hard-fail startup validation.** Every sensor/actuator handle is validated before the simulation runs a single real timestep. If any are invalid, the run stops immediately with a clear diagnostic rather than silently producing invalid data for days.

2. **Hard-coded, config-independent safety bounds.** Cooling/heating limits and the 2°C/cycle ramp-rate limit are hard-coded in `safety_validator.py`, independent of `settings.yaml` — a configuration file edit alone can never weaken a safety bound.

3. **Continuous occupancy-state enforcement.** Every zone's setpoint is re-validated against its current occupancy state every tick, independent of whether the model's tool calls happened to touch that zone — a setpoint can never silently outlive the occupancy state it was set for.

4. **Guaranteed audit trail.** `log_decision` is required every cycle. If the model's turn ends without calling it, the system writes an auditable entry itself, clearly flagged as automatic rather than model-authored.

5. **Fallback on outright LLM failure.** The last known-good setpoints are applied if an LLM call fails entirely, rather than leaving the building uncontrolled for that cycle.

6. **Per-tool error isolation.** A malformed or missing argument on any tool call fails only that call, not the entire decision cycle.

8. Testing & Verification Methodology

Every component was verified by actually running it against the real system — real EnergyPlus, the real local LLM, and (for final validation) a complete real 7-day simulation — rather than relying on code review or assumptions about correctness.

Phased Development & Verification

Phase	Scope	Verification Approach

0	Environment verification	Confirmed EnergyPlus, Ollama, and the target model all function together before writing project code.
1	Project skeleton & configuration	Verified the setup script resolves cleanly in a throwaway virtual environment.
2	EnergyPlus handle validation	Hard-fail validation tested by deliberately breaking a handle name to confirm the failure path actually triggers.
3	Sensor read loop	A full simulated day's sensor data checked for physical plausibility, not just absence of crashes.
4	Comfort & carbon models	PMV model cross-validated against an independent reference implementation across 7 conditions.
5	Safety validator	An occupancy-transition edge case proven numerically to violate safety bounds under the original design, then fixed and re-verified.
6–7	Tool registry & agent loop	All 4 tools exercised directly, then wired to the real LLM for the first time against hand-built scenarios.
8	Full integration	Real LLM + real EnergyPlus wired together; latency and reliability measured and improved based on real data.
9	Baseline, dashboard, full pipeline	Dashboard opened in an actual browser to catch issues invisible to unit-level checks.
Final	Real 7-day runs (baseline + ARIA)	Complete, non-truncated real-world evaluation; every downstream number traced back to this data.

Automated Test Suite

Test File	Tests	Coverage
`test_pmv.py`	11	PMV/PPD against cross-validated reference values and mathematical invariants
`test_safety.py`	17	Setpoint/lighting clamps, ramp-rate limits, occupancy-transition edge cases
`test_tools.py`	21	All 4 tools, single-zone and batched forms, malformed-argument handling
`test_database.py`	2	Timestep snapshot persistence and run-type filtering

`test_energyplus_env.py`	4	Occupancy-transition safety net, via mocked EnergyPlus objects
`test_agent.py`	5	The real Ollama + qwen2.5:3b tool-calling loop, including forced-failure handling
Total	**60**	All passing

9. Key Challenges Faced & Resolutions

Development surfaced a number of real, non-obvious issues — each caught by testing against the actual system rather than by assumption. The most significant are summarized below.

1. **A hard-fail that didn't actually fail.** A safety check's `raise RuntimeError(...)`, executed inside an EnergyPlus ctypes callback, was silently swallowed by ctypes — the simulation continued despite a fatal error. Fixed by setting a flag inside the callback and raising it for real in ordinary Python call context afterward.

2. **An API that doesn't implement its own documentation.** EnergyPlus's `request_variable()`, the documented way to register a sensor at runtime, registers nothing in this build — only variables statically declared in the building model file ever resolve. Every sensor ARIA reads was added to the model file directly.

3. **The most significant finding of the project:** the building model's `SimulationControl` setting for running the real weather-file period was disabled by default. This meant the real 7-day simulation had never executed even once in any earlier testing phase — every prior "real run" was unknowingly exercising EnergyPlus's throwaway sizing calculations instead. Caught by noticing a tick count that didn't add up, and fixed before any evaluation data was collected.

4. **A safety validator that could unclamp itself.** The original clamping order let a ramp-rate limit override the absolute safety bounds during an occupancy transition, which would have allowed an occupied zone's heating setpoint to sit 3°C below its safety floor. Proven numerically before it could occur in a live run, then fixed by re-applying the absolute clamp after the ramp-rate clamp.

5. **An LLM latency crisis, diagnosed and resolved with data.** One-tool-call-per-zone design produced a 52% cycle failure rate. Redesigning the tools to accept batched zone arrays, guided by measured cycle-time data, cut the failure rate to 19% — detailed in Section 5.

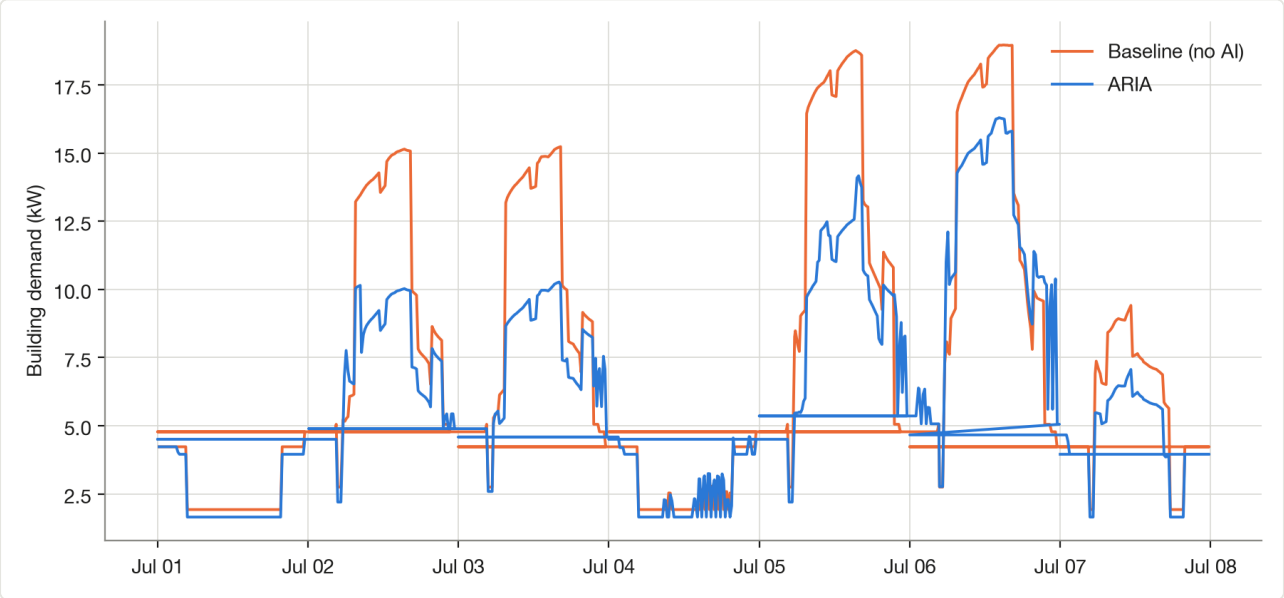
6. **A comfort shortfall investigated rather than hidden.** ARIA's occupied-zone PMV compliance (84.3%) is lower than the baseline's trivial 100%. Rather than reporting the number alone, the shortfall was traced to a specific, identified cause: 89% of violations are cold-side and cluster in the early-morning occupancy transition, consistent with carbon-driven precooling not fully backing off before zones become occupied — a real, honestly-reported energy-comfort tradeoff, detailed in Section 11.

7. **A production-hygiene bug found during final hardening.** A routine import of the baseline-run script executed a full simulation as a side effect and duplicated rows in the results database, because the script lacked a standard `if __name__ == "__main__":` guard present in the main entry point. Caught immediately, the duplicate data was verified and removed, and every entry-point script now follows the same guarded pattern.

10. The Real 7-Day Simulation & Results

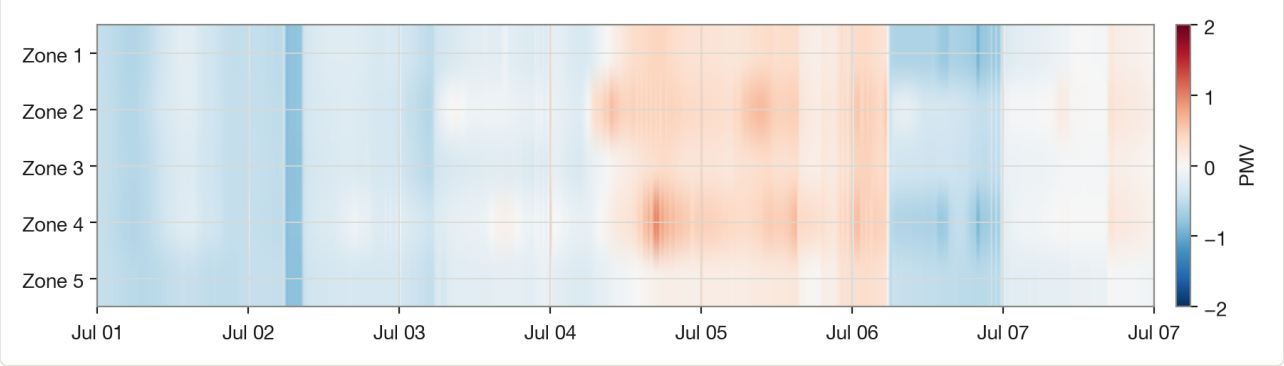
Both a no-AI baseline run and the full ARIA run were executed against the identical building model, weather file, and 7-day period (July 1–7), differing only in whether ARIA's control loop was active. The baseline run completed in under two seconds of EnergyPlus compute time (pure physics, no LLM calls); the ARIA run completed in approximately five hours of continuous real inference across 672 decision cycles.

Energy: ARIA vs. Baseline Building Demand



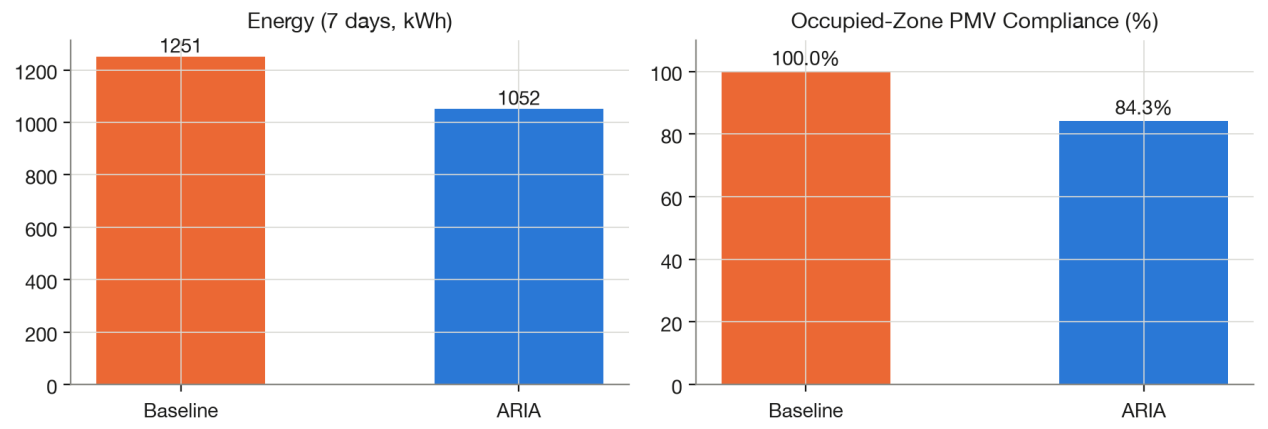
Building electrical demand (kW) over the full 7-day period. ARIA (blue) consistently reduces peak daytime demand relative to the static baseline schedule (orange).

Comfort: PMV by Zone Over Time (ARIA)



Predicted Mean Vote per zone across the full run. Near-white indicates thermal neutrality; blue/red indicate cold/hot deviation. The visible warm band (days 4–6) is discussed in Section 11.

Summary Comparison



11. Baseline vs. ARIA — Full Comparison

Metric	Baseline	ARIA	Change
Energy consumption (7 days)	1,251.2 kWh	1,051.8 kWh	↓ **15.9%**
Energy, occupied hours only	952.7 kWh	761.4 kWh	↓ **20.1%**
Estimated cost saved	—	~$29.90	@ $0.15/kWh (estimate)
Estimated CO_2 avoided	—	~43.7 kg	@ mock grid average (estimate)
Occupied-zone PMV compliance	100.0%	84.3%	see analysis below
Average occupied-zone PMV	~0.0 (fixed schedule)	-0.17	within [-0.5, +0.5] target
Absolute safety envelope violations	0	0	Never left 20–32°C / 15–24°C
Decisions logged	—	672 / 672	100% coverage
LLM-authored decisions	—	667 (99.3%)	5 auto-generated fallback (0.74%)

On the Comfort Number, Honestly

The no-AI baseline achieves 100% PMV compliance trivially: a static schedule is tuned to sit in the safe middle of the comfort band permanently, at the cost of exactly the energy ARIA exists to save. ARIA's 84.3% reflects it actually taking on that tradeoff, and the shortfall has a specific, identified cause rather than being unexplained: 233 of 261 occupied-zone violations (89%) were on the *cold* side, concentrated most heavily between 6–9am as occupancy begins. This lines up with the carbon-aware instruction to lower cooling toward 21°C during low-carbon overnight windows — that setpoint change does not always fully correct upward by the time a zone becomes occupied, producing a transient overcooling window rather than a sustained problem (the average occupied PMV across the entire run, -0.17, sits comfortably inside the [-0.5, +0.5] target).

> **Notably:** energy savings during occupied hours specifically (-20.1%) exceed the overall savings (-15.9%) — meaning the reduction is not merely a result of turning systems off when no one is present. It is partly a direct consequence of the same setpoint choices responsible for the comfort shortfall above. This is the real tradeoff the system made, stated plainly rather than obscured by an aggregate number.

12. Conclusion & Future Work

ARIA demonstrates a complete, working closed-loop building management agent: a real local LLM reasoning over a physics-accurate building simulation, safety-gated at every actuation, self-correcting against its own known failure modes, and producing a fully transparent audit trail of every decision it makes. Across a genuine, complete 7-day evaluation, it reduced energy consumption by 15.9% while keeping occupant comfort within its target band on average, and it did so with 99.3% of its decisions authored directly by a 3-billion-parameter model running entirely on local hardware.

Recommended Future Work

- Scope carbon-driven precooling instructions explicitly to unoccupied zones, or give the comfort target the same real-time occupancy-transition priority already built for the hard safety envelope, to close the comfort shortfall identified in Section 11.
- Extend the building model with real per-zone humidity and air-velocity sensing to replace the fixed comfort-model assumptions currently in use.
- Evaluate a larger open-source tool-calling model to quantify the reliability/latency tradeoff against qwen2.5:3b at scale.

- Extend the evaluation period beyond 7 days and across additional weather seasons.

13. Appendix — Tech Stack & Project Structure

Technology Stack

EnergyPlus · Ollama (qwen2.5:3b) · Python · Streamlit · Plotly · SQLite · pandas

Project Structure

Path	Purpose
`main.py`	Full ARIA run entry point (EnergyPlus + LLM agent)
`run_baseline.py`	No-AI reference run entry point
`agent/`	Reasoning loop, tool registry, safety validator, fallback handler, prompts
`simulation/`	EnergyPlus integration, handle registry, thread-safe state manager
`comfort/`	ISO 7730 PMV/PPD comfort model
`carbon/`	Grid carbon intensity (live API or mock)
`data/`	SQLite schema and read/write layer
`dashboard/`	Streamlit visualization application
`config/`	Central configuration and environment resolution
`tests/`	60-test automated suite
`docs/`	Architecture, testing log, and challenges documentation

Document prepared by Konepalli Harshavardhan
Honeywell Hackathon 2026 · Eco-Loop Building Agents Track